



Linux for Beginners

A Practical Guide

Chapter 11 of 12

System Logs, Services & systemd

Contents

11.1	Why systemd? From init Scripts to Units
11.2	Units and Targets: The Vocabulary of systemd
11.3	Controlling Services with systemctl
11.4	Enabling, Disabling, and Checking Status
11.5	Reading Logs the Modern Way: journalctl
11.6	Filtering journalctl by Time, Unit, and Priority
11.7	The Classic Log Files: /var/log in Depth
11.8	Keeping Logs Under Control: logrotate
11.9	Writing Your Own systemd Service Unit
11.10	Scheduling Recurring Tasks with cron and crontab
11.11	Cron Syntax Shortcuts and Special Strings
11.12	One-Time Scheduling with at
11.13	Catching Up Missed Jobs with anacron
11.14	Practical Scenarios: Logs, Services & Scheduling
11.15	Essential Commands: Quick Reference
11.16	Chapter Summary

Chapter 11

System Logs, Services & systemd

Every command you have typed so far has been reactive — you run it, it does something, and it's done. This chapter is where Linux starts doing things without you sitting at the keyboard. You'll meet **systemd**, the process manager that starts, stops, and supervises almost everything running on a modern Linux system; **journalctl** and the classic `/var/log` files, which record everything that happens; and **cron**, **at**, and **anacron**, the three ways Linux runs a task on a schedule instead of on demand. By the end of this chapter you'll be able to diagnose a service that won't start, read what happened during the last boot, and set up a task that runs itself every night at 2 a.m.

11.1 Why systemd? From init Scripts to Units

For decades, Linux distributions started services using **SysVinit**: a series of shell scripts in `/etc/init.d/`, run one after another in a fixed numeric order defined by symlinks in `/etc/rcN.d/`. It worked, but it was slow (everything started serially, waiting its turn) and fragile (dependencies between services were implicit in the script ordering, not declared anywhere the system could reason about).

systemd replaced this with a dependency graph of declarative *units*. Instead of a shell script that manually checks if a service is already running, a systemd unit simply declares what it needs and what it provides, and systemd figures out the fastest safe order to start everything — launching independent services in parallel rather than one at a time.

Concern	SysVinit (Old)	systemd (Modern)
Startup order	Serial, numbered scripts (S01, S02, ...)	Parallel, dependency-graph based
Dependencies	Implicit, encoded in script order	Explicit: <code>After=</code> , <code>Requires=</code> , <code>Wants=</code>
Service definition	Free-form shell script	Declarative unit file (INI-style)
Logging	Each service logs however it wants	Centralized via <code>journald</code>
On-demand starts	Not supported	Socket and D-Bus activation

```
$ ps -p 1 -o comm=
systemd
```

BASH

```
# PID 1 is always the very first process the kernel starts.
# On virtually every modern distribution, that process is systemd.
```

TIP

Ubuntu, Debian, Fedora, RHEL, and Arch all ship systemd as PID 1 by default today. Alpine and a handful of minimalist distributions are the well-known holdouts, using OpenRC instead.

11.2 Units and Targets: The Vocabulary of systemd

Everything systemd manages is a **unit**. A unit's type is given by its file extension, and each type manages a different kind of resource.

Unit Type	Extension	Manages
Service	.service	A long-running process, e.g. nginx or sshd
Socket	.socket	A network or IPC socket, can start its service on first connection
Mount	.mount	A filesystem mount point
Timer	.timer	A scheduled trigger — systemd's alternative to cron
Target	.target	A named group/synchronization point for other units
Path	.path	Triggers a unit when a file or directory changes

Targets replace the old numbered SysVinit "runlevels" with readable names. A target doesn't do anything itself — it's a checkpoint that groups the units that should be active at that stage of the boot.

Old Runlevel	systemd Target	Meaning
0	poweroff.target	Shut the machine down
1	rescue.target	Single-user maintenance mode
3	multi-user.target	Full multi-user system, no GUI (typical servers)
5	graphical.target	Multi-user system plus a graphical desktop
6	reboot.target	Restart the machine

```

$ systemctl get-default
graphical.target

$ systemctl list-units --type=service --state=running
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
nginx.service                      loaded active running A high performance web server
ssh.service                        loaded active running OpenBSD Secure Shell server
cron.service                       loaded active running Regular background program processing

```

11.3 Controlling Services with systemctl

`systemctl` is the single command you use to control, inspect, and query almost anything `systemd` manages.

```

$ sudo systemctl start nginx
$ sudo systemctl stop nginx
$ sudo systemctl restart nginx
$ sudo systemctl reload nginx

$ systemctl status nginx
* nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-08-29 08:12:04 IST; 2h 14min ago
     Docs: man:nginx(8)
  Main PID: 1042 (nginx)
    Tasks: 3 (limit: 4617)
   Memory: 6.8M
      CPU: 412ms
  CGroup: /system.slice/nginx.service
          |-1042 "nginx: master process /usr/sbin/nginx"
          `-1043 "nginx: worker process"

Aug 29 08:12:04 web01 systemd[1]: Starting nginx...
Aug 29 08:12:04 web01 systemd[1]: Started nginx.

```

Subcommand	What It Does
start	Starts a unit right now (does not survive a reboot by itself)
stop	Stops a running unit right now
restart	Stops then starts a unit — brief downtime
reload	Asks the service to reload its config without dropping connections

Subcommand	What It Does
status	Shows current state, recent log lines, and the process tree

TIP

reload is gentler than restart when it's supported — nginx and many other daemons can pick up a new config file without ever closing an active connection.

11.4 Enabling, Disabling, and Checking Status

start and stop are about right now. enable and disable are about the future — whether a service comes up automatically the next time the machine boots. The two are independent, which is a common source of confusion.

	Not Enabled	Enabled
Running now	Started manually this session only	Running now AND will start on next boot
Not running	Off, and stays off after a reboot	Off now, but will start on next boot

```

$ sudo systemctl enable nginx
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service ...

$ sudo systemctl enable --now nginx    # enable AND start in one command

$ systemctl is-enabled nginx
enabled
$ systemctl is-active nginx
active

$ sudo systemctl disable nginx
Removed /etc/systemd/system/multi-user.target.wants/nginx.service.
```

For something stronger than disable, mask links the unit to /dev/null, so it cannot be started even by accident or by another service's dependency:

```

$ sudo systemctl mask sendmail
$ sudo systemctl unmask sendmail
```

TIP

enable alone never starts anything immediately. If a service should be running right now AND after every future reboot, use 'enable --now'.

11.5 Reading Logs the Modern Way: journalctl

journald is systemd's logging service. It captures the standard output and error streams of every unit, kernel messages, and structured log entries, all in one indexed, binary journal — queried with **journalctl**.

```
$ journalctl                                     BASH
-- Journal begins at Mon 2026-08-24, ends at Sat 2026-08-29 --
Aug 29 08:12:04 web01 systemd[1]: Starting nginx...
Aug 29 08:12:04 web01 nginx[1042]: config file /etc/nginx/nginx.conf ok
Aug 29 08:12:04 web01 systemd[1]: Started nginx.

$ journalctl -e          # jump straight to the end, like less +G
$ journalctl -f          # follow new entries live, like tail -f
$ journalctl -r          # newest entries first
```

```
$ journalctl -u nginx                                     BASH
Aug 29 08:12:04 web01 systemd[1]: Starting nginx...
Aug 29 08:12:04 web01 systemd[1]: Started nginx.
Aug 29 10:03:41 web01 nginx[1042]: [error] 1043#1043: *4 open() failed
```

TIP

journalctl -u <unit> -f is the single most useful command for watching a service misbehave in real time.

11.6 Filtering journalctl by Time, Unit, and Priority

The journal can hold days or weeks of history, so filtering matters. **--since** and **--until** accept both natural language and exact timestamps.

```
$ journalctl --since "1 hour ago"                                     BASH
$ journalctl --since yesterday --until today
$ journalctl --since "2026-08-29 08:00:00"
$ journalctl -u nginx --since today
```

Priority filtering narrows results to a severity level and everything more severe than it:

Level	Name	Meaning
0	emerg	System is unusable
1	alert	Action must be taken immediately
2	crit	Critical condition
3	err	Error condition
4	warning	Warning condition
5	notice	Normal but significant
6	info	Informational
7	debug	Debug-level detail

```
$ journalctl -p err -b      # errors and worse, since the current boot
$ journalctl -b             # everything from the current boot
$ journalctl -b -1         # everything from the PREVIOUS boot
$ journalctl --list-boots  # list every boot the journal has recorded
```

BASH

The journal isn't unlimited — it's kept on disk and can grow large. Check and trim it with:

```
$ journalctl --disk-usage
Archived and active journals take up 412.0M in the file system.

$ sudo journalctl --vacuum-time=2weeks
$ sudo journalctl --vacuum-size=200M
```

BASH

11.7 The Classic Log Files: /var/log in Depth

Even with `journald` front and center, plain text files under `/var/log` are still where many tools, and many admins' muscle memory, look first.

File	Contains
<code>/var/log/syslog</code>	General system activity log (Debian/Ubuntu)
<code>/var/log/messages</code>	General system activity log (RHEL/CentOS/Fedora equivalent)
<code>/var/log/auth.log</code>	Authentication attempts, sudo usage, SSH logins (Debian/Ubuntu)

File	Contains
/var/log/secure	Authentication log equivalent on RHEL-based systems
/var/log/kern.log	Kernel-level messages and warnings
/var/log/dmesg	Kernel ring buffer — boot-time hardware detection messages
/var/log/dpkg.log	Package install/remove/update history (Debian/Ubuntu)
/var/log/nginx/	Per-application logs, in this case nginx's access and error logs

```
$ tail -f /var/log/auth.log
Aug 29 11:02:17 web01 sshd[8842]: Accepted publickey for ashik from 203.0.113.9
Aug 29 11:04:03 web01 sudo: ashik : COMMAND=/usr/bin/apt update

$ grep "Failed password" /var/log/auth.log | tail -5
```

BASH**TIP**

On systems that keep both, dmesg and journalctl -k show the same kernel ring buffer — use whichever you have muscle memory for.

11.8 Keeping Logs Under Control: logrotate

Left alone, log files grow forever. Logrotate runs daily via cron or a systemd timer, and rotates, compresses, and eventually deletes old logs according to per-application rules.

The main config is /etc/logrotate.conf, but most applications drop their own rule file into /etc/logrotate.d/ so packages can manage their own log policy independently.

```
$ cat /etc/logrotate.d/nginx
/var/log/nginx/*.log {
    weekly
    rotate 12
    compress
    delaycompress
    missingok
    notifempty
    create 0640 www-data adm
    sharedscripts
    postrotate
        systemctl reload nginx > /dev/null 2>&1 || true
    endscript
}
```

BASH

Directive	Meaning
weekly / daily / monthly	How often to rotate
rotate N	Keep N old rotated copies before deleting the oldest
compress	Gzip rotated logs to save space
missingok	Don't error if the log file doesn't exist
notifempty	Don't rotate an empty log file
create MODE OWNER GROUP	Create a fresh, empty log file with these permissions after rotating

```
$ sudo logrotate -d /etc/logrotate.conf # dry run – shows what WOULD happen
$ sudo logrotate -f /etc/logrotate.conf # force rotation right now
```

BASH

TIP

Always test a new logrotate rule with -d first. A typo in the postrotate script silently breaks log rotation for that app.

11.9 Writing Your Own systemd Service Unit

Turning any script into a proper, supervised, auto-restarting background service is one unit file away. Here's a complete, working example for a small Python app:

```
$ sudo nano /etc/systemd/system/myapp.service
```

INI

```
[Unit]
Description=My Flask application
After=network.target

[Service]
Type=simple
User=ashik
WorkingDirectory=/opt/myapp
ExecStart=/usr/bin/python3 /opt/myapp/app.py
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Directive	Section	Meaning
Description	[Unit]	Human-readable summary shown in systemctl status
After	[Unit]	Start this unit only after the named unit is up
ExecStart	[Service]	The exact command that starts the service
User	[Service]	Run as this user instead of root
Restart	[Service]	on-failure, always, or no — auto-restart policy
WantedBy	[Install]	Which target's "wants" this unit joins when enabled

```
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now myapp
$ systemctl status myapp
```

BASH

WARNING

systemd only reads unit files at startup or when told to. After creating or editing any file under `/etc/systemd/system/`, you must run `'sudo systemctl daemon-reload'` or your changes are silently ignored.

11.10 Scheduling Recurring Tasks with cron and crontab

cron is the classic Unix task scheduler, run by the `crond` (or `cron`) daemon. Each user has their own schedule, called a *crontab*, edited with `crontab -e` rather than by hand.

```
$ crontab -e    # opens your personal crontab in $EDITOR
$ crontab -l    # list your current crontab
$ crontab -r    # remove your entire crontab (careful!)
```

BASH

Every line follows the same five-field time pattern, followed by the command to run:

Field	Allowed Values	Meaning
1. Minute	0-59	Minute of the hour
2. Hour	0-23	Hour of the day (24-hour clock)
3. Day of month	1-31	Day of the month
4. Month	1-12	Month of the year
5. Day of week	0-7 (0 and 7 = Sunday)	Day of the week

```
# minute hour dom month dow  command
0      2    *    *    *    /usr/local/bin/backup.sh
*/15   *    *    *    *    /usr/local/bin/check-disk.sh
30     6    *    *    0    /usr/local/bin/weekly-report.sh
```

CRON

Line one runs a backup script at 2:00 a.m. every day. Line two runs a disk check every 15 minutes, all day, every day. Line three runs a report every Sunday at 6:30 a.m.

11.11 Cron Syntax Shortcuts and Special Strings

Symbol	Meaning	Example
*	Every value	* * * * * runs every minute
*/N	Every N units	*/5 * * * * runs every 5 minutes
A-B	A range	1-5 in the weekday field means Mon-Fri
A,B,C	A specific list	1,15 in the day field means the 1st and 15th

For common schedules, cron also accepts readable special strings instead of the five-field pattern:

Special String	Equivalent To
@reboot	Run once, at startup
@hourly	0 * * * *

Special String	Equivalent To
@daily	0 0 * * *
@weekly	0 0 * * 0
@monthly	0 0 1 * *
@yearly	0 0 1 1 *

Beyond individual crontabs, the system-wide `/etc/crontab` and drop-in files under `/etc/cron.d/` add one extra field — the user to run as — since they aren't owned by a single person:

```
# /etc/crontab – note the extra 'user' field before the command
# min hour dom month dow user  command
0 3 * * * root /usr/local/bin/system-cleanup.sh
```

CRON

Ubuntu and Debian also provide `/etc/cron.daily/`, `cron.hourly/`, `cron.weekly/`, and `cron.monthly/` directories — anything executable dropped in one of these runs on that cadence automatically, via `run-parts`.

TIP

Cron jobs run with a minimal environment — no `$PATH` beyond the basics, no interactive shell config. Always use full paths in scripts, and redirect output to a log file so failures aren't silently lost.

11.12 One-Time Scheduling with at

`cron` is for recurring jobs. When you need something to run exactly once, at a specific future time, `at` is the right tool.

```
$ sudo apt install at
$ sudo systemctl enable --now atd

$ at 15:00
at> /usr/local/bin/send-reminder.sh
at> <EOT>
job 3 at Sat Aug 29 15:00:00 2026

$ echo "/usr/local/bin/cleanup.sh" | at now + 10 minutes
$ at now + 1 hour <<< "/usr/local/bin/reconnect-vpn.sh"
```

BASH

Time Expression	Meaning
now + 10 minutes	10 minutes from now
15:00	3:00 PM today (or tomorrow, if already past)
tomorrow	9:00 AM tomorrow by default
4pm + 3 days	4:00 PM, three days from now

```
$ atq
3    Sat Aug 29 15:00:00 2026 a ashik

$ atrm 3      # cancel job number 3
```

BASH

11.13 Catching Up Missed Jobs with anacron

cron assumes the machine is always on. If it's powered off at 2:00 a.m. when a backup job was scheduled, that job simply never runs — cron doesn't remember and doesn't retry. That's a real problem on laptops and desktops that aren't running around the clock. **anacron** fixes this by tracking, in days, how long it's been since a job last ran, and firing it as soon as the machine is next on if it's overdue.

```
$ cat /etc/anacrontab
# period delay job-identifier      command
1      5      cron.daily          run-parts /etc/cron.daily
7      25     cron.weekly         run-parts /etc/cron.weekly
30     45     cron.monthly        run-parts /etc/cron.monthly
```

BASH

Field	Meaning
period	How often, in days, this job should run (1 = daily, 7 = weekly)
delay	Minutes to wait after boot before running, so the system isn't overloaded
job-identifier	A unique name anacron uses to track when the job last completed
command	The command to execute — typically run-parts on a cron.* directory

In practice, most desktop distributions already wire `cron.daily`, `cron.weekly`, and `cron.monthly` through **anacron** out of the box. The rule of thumb: **cron** for jobs that must run at an exact time on an always-on server; **anacron** for jobs that just need to happen eventually on a machine that sleeps.

11.14 Practical Scenarios: Logs, Services & Scheduling

Scenario 1: A service refuses to start

1. Run `systemctl status` first — it shows the exact failure and the last few log lines in one place.
2. For the full history, follow up with `journalctl -u -e`.
3. Check the unit file itself for typos: `systemctl cat .`
4. After fixing a unit file, always run `systemctl daemon-reload` before trying again.

Scenario 2: A disk is filling up with logs

1. Check how much space the journal itself is using: `journalctl --disk-usage`.
2. Trim it if needed: `sudo journalctl --vacuum-time=2weeks`.
3. Check for an application logging outside logrotate's reach: `du -sh /var/log/* | sort -rh`.
4. Add or fix a logrotate.d rule for the offending application, then test it with `logrotate -d`.

Scenario 3: Setting up a reliable nightly backup

1. Write and test the backup script manually first, confirming it works end to end.
2. Add a line to your crontab: `0 2 * * * /usr/local/bin/backup.sh >> /var/log/backup.log 2>&1`.
3. Redirect both stdout and stderr to a log file — cron only emails output if mail is configured, and often it isn't.
4. Check `/var/log/backup.log` the next morning to confirm it actually ran.

Scenario 4: Investigating what happened at boot

1. `journalctl -b` shows every log line from the current boot, start to finish.
2. `journalctl -b -1` shows the previous boot — essential after an unexpected reboot or crash.
3. `journalctl -p err -b` filters that same boot down to just errors and worse.
4. `systemd-analyze blame` ranks every service by how long it took to start, which is the fastest way to find what's slowing down boot.

11.15 Essential Commands: Quick Reference

Command	What It Does	Common Usage
<code>systemctl start</code>	Starts a unit immediately	<code>sudo systemctl start nginx</code>
<code>systemctl stop</code>	Stops a running unit	<code>sudo systemctl stop nginx</code>
<code>systemctl restart</code>	Stops then starts a unit	<code>sudo systemctl restart nginx</code>

Command	What It Does	Common Usage
<code>systemctl status</code>	Shows state, PID, and recent logs	<code>systemctl status nginx</code>
<code>systemctl enable</code>	Starts a unit automatically on future boots	<code>sudo systemctl enable nginx</code>
<code>systemctl enable --now</code>	Enables AND starts in one step	<code>sudo systemctl enable --now nginx</code>
<code>systemctl disable</code>	Removes a unit from auto-start	<code>sudo systemctl disable nginx</code>
<code>systemctl mask</code>	Blocks a unit from being started at all	<code>sudo systemctl mask sendmail</code>
<code>systemctl daemon-reload</code>	Reloads unit files after editing them	<code>sudo systemctl daemon-reload</code>
<code>systemctl list-units</code>	Lists active units	<code>systemctl list-units --type=service</code>
<code>journalctl</code>	Shows the full systemd journal	<code>journalctl -e</code>
<code>journalctl -u</code>	Filters the journal to one unit	<code>journalctl -u nginx -f</code>
<code>journalctl -b</code>	Filters to a specific boot	<code>journalctl -b -1</code>
<code>journalctl -p</code>	Filters by minimum priority	<code>journalctl -p err</code>
<code>journalctl --since</code>	Filters by start time	<code>journalctl --since yesterday</code>
<code>logrotate -d</code>	Dry-runs a logrotate config	<code>sudo logrotate -d /etc/logrotate.conf</code>
<code>logrotate -f</code>	Forces rotation right now	<code>sudo logrotate -f /etc/logrotate.conf</code>
<code>crontab -e</code>	Edits your personal schedule	<code>crontab -e</code>
<code>crontab -l</code>	Lists your personal schedule	<code>crontab -l</code>
<code>at</code>	Schedules a one-time future job	<code>at now + 10 minutes</code>
<code>atq</code>	Lists pending at jobs	<code>atq</code>
<code>atrm</code>	Cancels a pending at job	<code>atrm 3</code>
<code>systemd-analyze blame</code>	Ranks services by boot startup time	<code>systemd-analyze blame</code>

11.16 Chapter Summary

- ✓ systemd replaced sequential SysVinit scripts with a parallel, dependency-aware graph of declarative units.
- ✓ `systemctl start/stop/restart` act now; `enable/disable` decide what happens on future reboots — the two are independent.

- ✓ `systemctl status` is the fastest way to see whether a service is healthy and why it might not be.
- ✓ `journalctl` replaces (and complements) classic log files, filterable by unit, time, boot, and priority.
- ✓ `/var/log/syslog`, `auth.log`, and `kern.log` are still the first place many admins look for plain-text history.
- ✓ `logrotate` keeps logs from growing forever, rotating and compressing them on a schedule per application.
- ✓ A `systemd .service` unit turns any script into a supervised, auto-restarting background service — after a `daemon-reload`.
- ✓ `cron` runs recurring jobs at exact times using a five-field schedule; `at` runs a job exactly once in the future.
- ✓ `anacron` catches up on missed `cron.daily/weekly/monthly` jobs on machines that aren't always powered on.

What's Coming in Chapter 12: What's Next: Linux in the Real World

- A recap of everything covered across all 12 chapters, and how the pieces fit together
- Where these Linux skills apply in the real world: `sysadmin`, `DevOps`, cybersecurity, cloud, and development roles
- Recommended next topics: Docker, Kubernetes, Ansible, Terraform, and Python automation
- Certifications worth considering: LPIC-1, CompTIA Linux+, and RHCSA
- How to keep learning: man pages, official docs, and communities worth joining
- Setting up a home lab to practice safely
- Contributing to open source as a way to keep building real skills
- Closing words on how this guide came together

You've gone from asking "what is Linux?" to running, monitoring, and scheduling your own services on it. The final chapter is about where you take that next.

Author

Ashik A

github.com/ashihh07

This guide was created, organized, refined, and designed by Ashik A using GitHub documentation, industry best practices, publicly available learning resources, and AI-assisted research.
